



Hirkanet

Complete Application, Deployment, Operations and Usage Guide

Network and Application Security Platform Secure Track FortiGate Policy Evaluator

Based on the cumulative project build through remediation #28
31 July 2026

Scope: This manual describes the supplied project code and configuration. Commands must be reviewed for the target host, domain, credentials, network, and organizational change-control requirements.

Document control

Field	Value
Document	Hirkanet Complete Deployment, Operations and Usage Guide
Project basis	hirkanet-fixed-28-deployment-database-docs
Primary audience	System administrators, security engineers, developers, database operators and service owners
Deployment model	Separate application/PostgreSQL and Elastic Compose projects
Canonical public domain	https://hirkanet.com
Security service	Secure Track FortiGate Policy Evaluator
Database	PostgreSQL in Docker; SQLite supported for local development
Change discipline	Back up first, apply migrations, test, monitor, and maintain rollback procedures

Important: The policy evaluator ranks candidate policies heuristically. It is not a complete FortiGate packet-flow simulator and must not be used as authoritative proof of traffic allowance, denial, routing, NAT, or reachability.

How to use this guide

- New deployment: read Chapters 1, 2, 8, 9 and 10 in order, then use the deployment checklist.
- Application operator: focus on Chapters 3, 4, 5, 11 and 13.
- Database operator: focus on Chapters 6 and 12.
- Security/observability operator: focus on Chapters 7, 9 and 14.
- Developer: focus on Chapters 2, 3, 4, 5, 6, 15 and the appendices.

Contents

- [1. Platform overview](#)
- [2. Repository and module architecture](#)
- [3. Application architecture and request lifecycle](#)
- [4. Secure Track evaluation engine](#)
- [5. Validation and application security](#)
- [6. Database design and lifecycle](#)
- [7. Logging and Elastic observability](#)
- [8. Pre-deployment preparation](#)
- [9. Docker deployment](#)
- [10. HTTPS and public-domain deployment](#)
- [11. User and administrator guide](#)
- [12. Database operations](#)
- [13. FortiGate snapshot operations](#)
- [14. Monitoring, retention and incident response](#)
- [15. Development, testing and change management](#)
- [16. Troubleshooting](#)
- [Appendix A. Command reference](#)
- [Appendix B. Configuration reference](#)
- [Appendix C. Security and production checklist](#)

1. Platform overview

Hirkanet is a network and application security platform. The current project combines a public website, authenticated client portal, administrative portal, PostgreSQL-backed identity and content management, a FortiGate configuration collector, the Secure Track policy evaluator, and an optional Elastic observability stack.

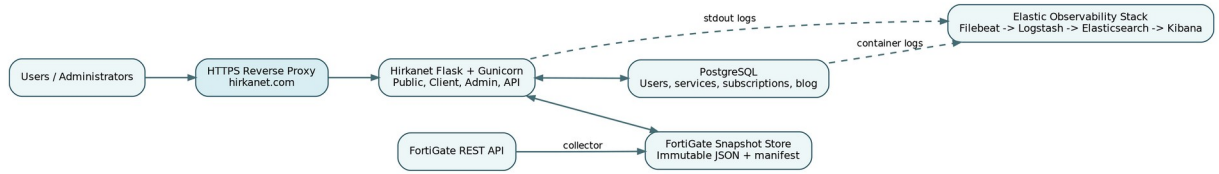


Figure 1 - Hirkanet high-level architecture

1.1 Core capabilities

- Public website and blog content managed from the administrative portal.
- Role-based authentication for administrators and clients.
- Service catalog and user subscriptions controlling client access.
- Secure Track evaluation of source, destination and service requests against collected FortiGate policy data.
- Interactive single-request and CSV batch evaluation workflows.
- Immutable, checksummed and versioned FortiGate data snapshots.
- Structured JSON logging with request IDs and sensitive-field redaction.
- Optional TLS-protected Filebeat, Logstash, Elasticsearch and Kibana deployment.
- Database migrations, backup/restore testing, storage monitoring and retention tooling.

1.2 Evaluation scope

The evaluator compares requested IPv4 endpoints and services with collected FortiGate objects, routes, interfaces and enabled accept policies. It calculates coverage and proximity indicators, applies least-privilege penalties, and returns ranked policy candidates with missing elements and suggested actions.

Not a simulator: The engine does not fully model policy order, explicit deny interactions, schedules, NAT or central SNAT, identity policies, zones, Internet Service objects, dynamic objects, policy routes, session state, multi-VDOM behavior or downstream reachability. Every result requires engineering review and normal change control.

1.3 Deployment boundaries

Boundary	Contents	Start command
Application stack	PostgreSQL, migration job, Hirkanet application	<code>docker compose up -d --build</code>
Elastic stack	Certificate bootstrap, Elasticsearch, initialization, Logstash, Kibana, Filebeat	<code>docker compose -f docker-compose.elastic.yml up -d</code>
Public edge	DNS, firewall and HTTPS reverse proxy	Managed outside the application Compose project

Boundary	Contents	Start command
FortiGate collection	Collector process with restricted FortiGate REST token	Run manually or via controlled scheduler

2. Repository and module architecture

The project uses Flask blueprints and separates public, client, administrative, authentication, evaluation, validation, persistence and operational tooling. Generated Python caches and local secret files are not architectural components and must not be committed.

```

hirkanet.com/
├── api/                # Flask application factory and global routes
├── main/              # Public home, blog and error routes
├── auth/              # Login/profile forms
├── client/            # Client dashboard and policy evaluation
├── admin/             # User, service, subscription and blog administration
├── engine/            # Evaluator, CIDR/range logic, routes and snapshots
├── validation/        # Centralized input validators
├── collectors/        # FortiGate REST collector
├── migrations/        # Alembic migration history
├── scripts/           # Backup, restore, storage monitoring and retention
├── elk/               # Filebeat, Logstash, ILM and CA rotation
├── deployment/nginx/  # Production reverse-proxy example
├── templates/         # Public, client and admin Jinja templates
├── static/            # CSS, logos and blog uploads
├── data/              # Active pointer and immutable FortiGate snapshots
├── docker-compose.yml
├── docker-compose.elastic.yml
└── Dockerfile

```

2.1 Flask modules

Module	Responsibility
api/app.py	Creates the Flask application, initializes SQLAlchemy, migrations, login, CSRF, logging, proxy trust, blueprints, health endpoints and CLI commands.
main/routes.py	Public home page, blog listing and blog detail routes.
client/routes.py	Subscription-aware client dashboard, policy evaluation, CSV batch processing and result presentation.
admin/routes.py	Users, subscriptions, services, categories, posts and administrative profile operations.
auth/forms.py	Flask-WTF login and password-change forms.

2.2 Core service modules

Module	Responsibility
models.py	SQLAlchemy models, constraints, relationships and database URL construction.
database_transactions.py	Centralized commit, rollback, logging and safe error behavior.
security.py	Password hashing and verification.

Module	Responsibility
secret_utils.py	Reads mounted secret files and optional environment fallbacks.
logging_config.py	JSON events, redaction, request IDs and HTTP access events.
engine/evaluator.py	Loads validated snapshots and ranks FortiGate policy candidates.
engine/snapshot_store.py	Active snapshot resolution and manifest/checksum verification.
validation/*	Reusable validators for users, admin forms, evaluation, CSV and images.

2.3 Operational assets

- docker-compose.yml controls the application database, migration job and web service.
- docker-compose.elastic.yml controls the independent observability stack.
- migrations/versions contains the baseline schema, foreign-key actions, constraints and indexes.
- scripts/postgres-*.sh provides backup, restore testing and guarded recovery.
- scripts/storage-monitor.py and scripts/storage-retention.py implement storage alerts and safe retention.
- elk/rotate-elastic-ca.sh requires explicit confirmation for trust-root rotation.

3. Application architecture and request lifecycle

3.1 Application initialization

1. Load non-secret configuration from environment variables and secret values from mounted files.
2. Configure structured logging before serving requests.
3. Initialize SQLAlchemy and verify database connectivity; schema creation is not performed by the web process.
4. Register Flask-Migrate, LoginManager and global CSRF protection.
5. Apply trusted-proxy and trusted-host handling when explicitly configured.
6. Register the public, client and administrative blueprints.
7. Expose /livez for process liveness and /readyz for database-backed readiness.

3.2 HTTP request lifecycle

8. The reverse proxy terminates HTTPS and forwards one trusted set of X-Forwarded headers.
9. Flask validates the host and reconstructs the external HTTPS scheme when proxy trust is enabled.
10. A request ID is accepted or generated and included in response headers and logs.
11. Authentication and authorization decorators enforce login, role and subscription requirements.
12. State-changing forms are protected by global CSRF validation.
13. Central validators normalize form or API input before domain operations.
14. Database writes execute through the shared transaction boundary.
15. The HTTP completion event records status, duration, request ID and user ID.

3.3 Authentication and authorization

Users are stored only in PostgreSQL. The legacy JSON user store has been removed. Password hashes are generated by the project security module; successful legacy-password verification can upgrade a hash within a transaction. Roles are constrained to admin or client at both validation and database levels.

Control	Implementation
Authentication	Flask-Login session authentication using the users table.
CSRF	Flask-WTF CSRFProtect across all state-changing forms.
Session cookie	HttpOnly, SameSite=Lax and Secure in production.
Host control	TRUSTED_HOSTS configuration.
Proxy trust	ProxyFix enabled only when TRUST_PROXY_HEADERS=true.
Client authorization	Active user subscription joined to an active service.
Admin authorization	Role check for admin routes.

4. Secure Track evaluation engine

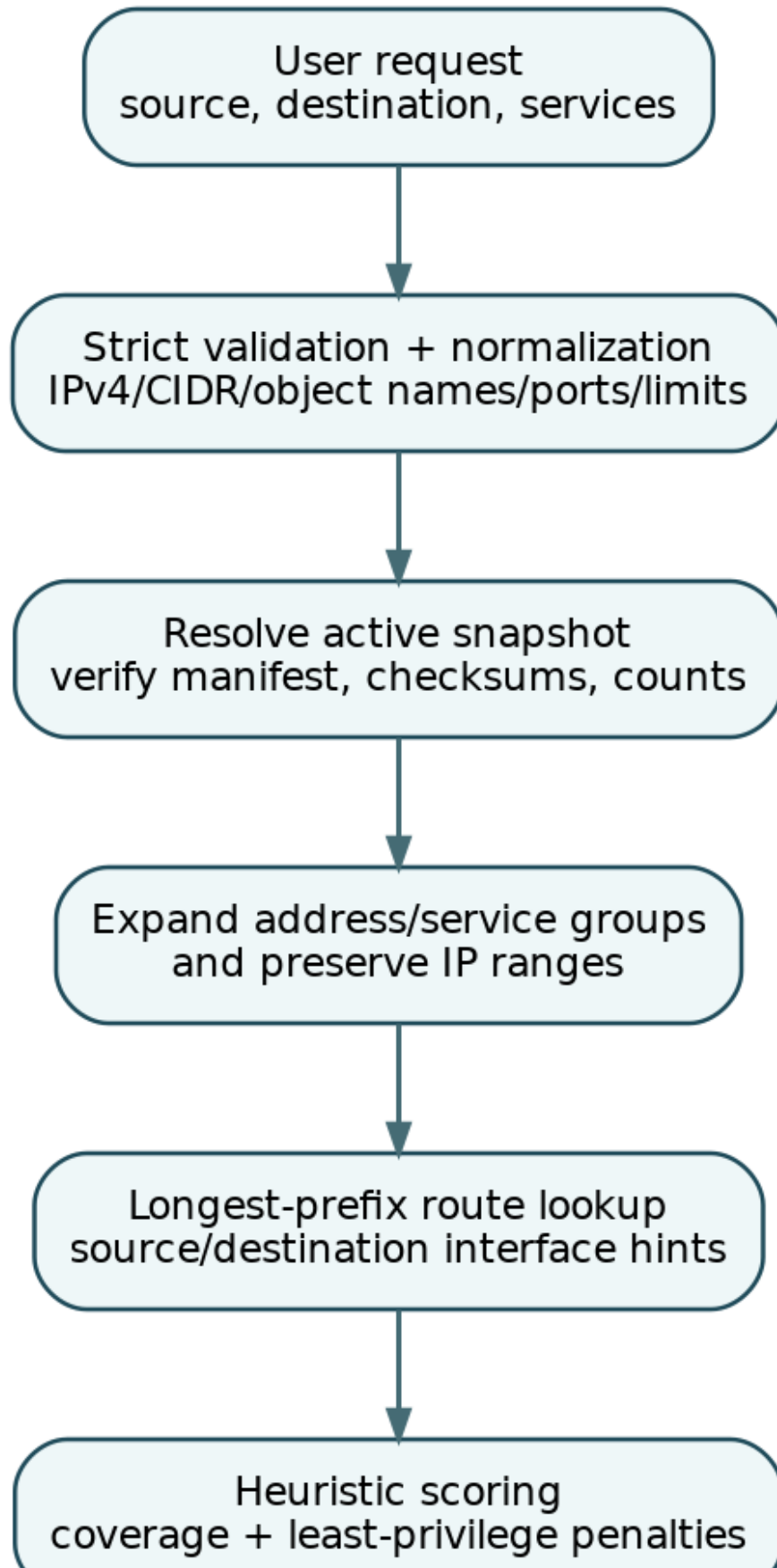


Figure 2 - Evaluation request lifecycle

4.1 Input model

A request contains one or more source addresses, destination addresses and services. Addresses may be IPv4 hosts, IPv4 CIDRs or known FortiGate address-object names. Services may be known FortiGate service names, numeric ports or normalized TCP/UDP expressions.

```
Sources:      192.168.10.25, 192.168.20.0/24, HR-CLIENTS
Destinations: 10.10.10.15, APP-SERVERS
Services:     443, tcp-8443, HTTPS
```

4.2 Strict validation

Rule	Current behavior
Address family	IPv4 supported; IPv6 rejected explicitly.
Known objects	Unknown non-IP address or service names are rejected.
Ports	Must be 1-65535; descending ranges are rejected.
List size	Maximum 50 values per field.
Value size	Maximum 128 characters per value.
Batch size	Maximum 1,000 data rows and 2 MiB CSV upload.
Deduplication	Case-insensitive duplicate removal.
Required fields	Source and destination are required.

4.3 Snapshot loading and integrity

The evaluator resolves data/current.json, loads exactly one immutable snapshot directory, verifies each required file against manifest sizes, record counts and SHA-256 checksums, and reloads when the active snapshot ID changes. This prevents mixed-version reads and partially updated policy data.

4.4 Address and service matching

- Hosts are normalized as /32 networks.
- CIDR containment is evaluated with Python ipaddress semantics.
- FortiGate address and service groups are expanded recursively.
- IP ranges retain both endpoints and use inclusive full-coverage semantics.
- A requested subnet or range matches only when the policy covers the entire requested interval.
- TCP/UDP service objects and ranges are normalized before comparison.

4.5 Route and interface hints

The interface selector performs longest-prefix matching against the snapshot routing table. The result is a routing hint used in scoring. Source-interface inference through reverse route lookup is an approximation and must not be treated as authoritative FortiGate ingress determination.

4.6 Scoring model

Signal	Typical effect
Source coverage	Positive score for each covered source request.
Destination coverage	Positive score for each covered destination request.
Service coverage	Positive score for each covered service request.
Interface alignment	Positive or negative score based on route-derived hints.
Wide objects	Penalties for all, /0 and broader-than-required objects.
Excess objects/services	Least-privilege penalties for unrelated rule elements.
Address proximity	Small heuristic points for near network relationships when not fully covered.

4.7 Interpreting results

Required review: A top-ranked candidate is a recommendation for investigation. Review policy order, deny rules, NAT, schedules, zones, VDOMs, routes, identities and downstream controls on the FortiGate before implementation.

- Already covered means the collected candidate appears to cover the validated request under the evaluator model.
- Add srcaddr, add dstaddr or add service indicates missing elements for a possible extension candidate.
- A high score does not prove runtime allowance or reachability.
- No result should bypass peer review, testing or rollback preparation.

5. Validation and application security

5.1 Centralized validation package

File	Purpose
validation/common.py	Shared text, identifier, checkbox, username, password and service-type rules.
validation/admin.py	Typed validation for users, services, subscriptions, categories and posts.
validation/evaluation.py	Strict evaluator address, service and batch constraints.
validation/uploads.py	CSV size/encoding and image signature validation.
validation/errors.py	Structured field-level ValidationError.

5.2 Upload security

- CSV uploads require a .csv extension, UTF-8 encoding and a 2 MiB maximum.
- Blog cover images are limited to PNG, JPG/JPEG, GIF or WEBP and checked by file signature.
- Upload filenames are normalized and generated safely rather than trusted directly.
- The writable static upload path is isolated from read-only application code in the hardened container.
- Uploaded media is monitored but never blindly deleted by automated retention.

5.3 Transaction safety

All application writes use database_transactions.py. A successful operation commits once. Integrity violations or other SQLAlchemy failures trigger rollback before the scoped session is reused. Constraint conflicts return a safe 409 response; other database failures return a safe 500 response without SQL details.

```
try:
    commit_transaction("create_user")
except DatabaseConflictError:
    # 409: no changes were saved
except DatabaseOperationError:
    # 500: session has already been rolled back
```

5.4 Container hardening

Control	Application setting
Non-root execution	Dedicated app user in the Dockerfile.
Read-only root filesystem	read_only: true.
Privilege escalation	no-new-privileges:true.
Linux capabilities	cap_drop: ALL.
PID control	pids_limit: 256.
/tmp	64 MiB tmpfs with noexec,nosuid,nodev.

Control	Application setting
Writable storage	Only data, uploads and static/uploads mounts.
Graceful shutdown	init: true and 30-second stop grace period.

5.5 Secrets

Passwords and encryption keys are mounted as files under `/run/secrets`. The `.env` file is reserved for non-secret settings. Secret files must be extensionless, excluded from Git, mode 0600 and readable only by the deployment account and the intended containers.

6. Database design and lifecycle

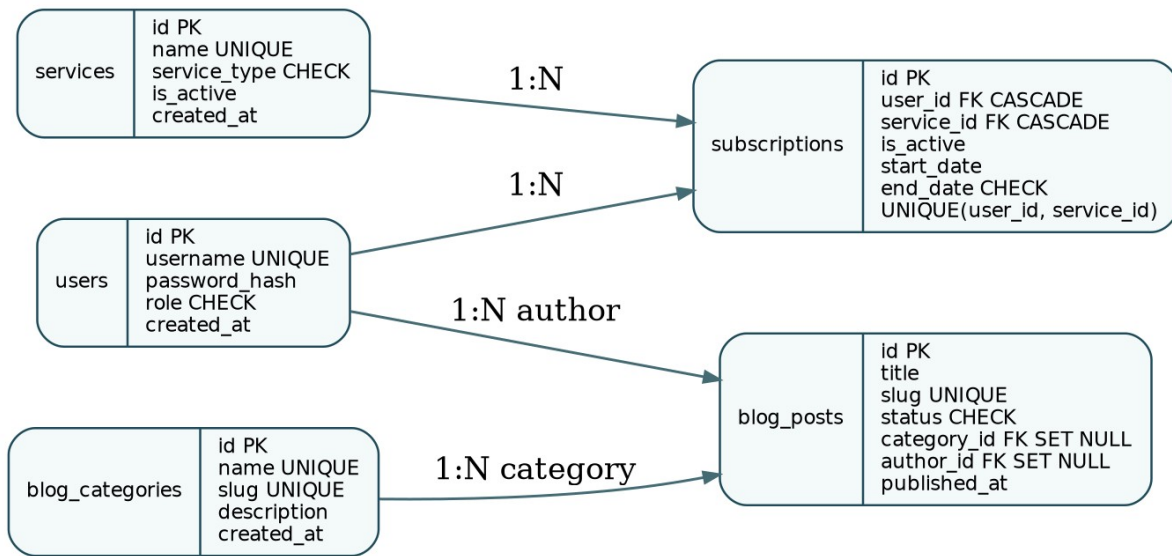


Figure 3 - PostgreSQL entity relationships and delete actions

6.1 Entity model

Entity	Purpose
users	Authentication identity, password hash, role and creation time.
services	Hirkanet service catalog; the policy evaluator is a service type.
subscriptions	User-to-service entitlement and optional date range.
blog_categories	Public content classification.
blog_posts	Draft or published content with optional author and category.
alembic_version	Tracks applied migration revision.

6.2 Constraints and deletion behavior

- User role is restricted to admin or client.
- Blog status is restricted to draft or published.
- Service type cannot be blank.
- A user can have only one subscription row per service.
- A subscription end date must be later than its start date.
- Deleting a user or service cascades to subscriptions.
- Deleting an author or category preserves blog posts and sets the foreign key to NULL.

6.3 Query indexes

Index	Supports
ix_services_type_active	Active service discovery by type.
ix_subscriptions_user_active	Client entitlement checks.
ix_subscriptions_service_active	Service administration and membership queries.
ix_blog_posts_status_published_at	Published post listing and ordering.
ix_blog_posts_category_status	Category-filtered public blog queries.

6.4 Database URL and credentials

Compose passes POSTGRES_HOST, POSTGRES_PORT, POSTGRES_DB and POSTGRES_USER separately. The application reads the password from a secret file and creates a SQLAlchemy URL with URL.create(). This safely handles reserved characters in passwords and avoids constructing DATABASE_URL in Compose.

6.5 Migration model

The web application does not call db.create_all(). A one-shot migrate service waits for PostgreSQL, runs flask db upgrade, then runs the idempotent seed-defaults command. Gunicorn starts only after the migration service succeeds.

```

PostgreSQL healthy
  |
  v
flask --app api.app db upgrade
  |
  v
flask --app api.app seed-defaults
  |
  v
Gunicorn application starts

```

Backup first: Take and test a PostgreSQL backup before applying a new migration in production. Review generated revisions; never auto-generate schema changes during container startup.

7. Logging and Elastic observability



Figure 4 - Structured logging and Elastic data path

7.1 Application events

Hirkanet writes structured JSON to stdout. Events include service name, dataset, level, request ID, HTTP method/path/status, duration, user ID where available, event type and structured exception details. Sensitive keys such as password, token, authorization, cookie, secret and csrf_token are redacted.

```
{
  "@timestamp": "2026-07-31T12:00:00+00:00",
  "message": "Policy evaluation completed",
  "log": {"level": "info", "logger": "client.routes"},
  "service": {"name": "hirkanet"},
  "event": {"dataset": "hirkanet.application"},
  "request": {"id": "..."},
  "event_type": "policy_evaluation_completed"
}
```

7.2 Independent Elastic stack

The observability stack is optional and runs in docker-compose.elastic.yml. It does not share a Docker network with the application stack. Filebeat reads Docker host logs and selects Hirkanet application and PostgreSQL containers through stable hirkanet_log_role labels.

7.3 TLS and identities

Connection	Hostname verification
Filebeat to Logstash	TLS, full verification, DNS SAN logstash.
Logstash to Elasticsearch	HTTPS, full verification, DNS SAN elasticsearch.
Kibana to Elasticsearch	HTTPS, full verification, DNS SAN elasticsearch.
Elasticsearch transport	TLS with full verification.
Browser to Kibana	HTTPS on loopback; trust the private CA in controlled environments.

7.4 ILM and dead-letter retention

Alias / policy	Rollover	Deletion
hirkanet-logs / hirkanet-logs-policy	10 GiB primary shard or 1 day	30 days after rollover
hirkanet-dead-letter / hirkanet-dead-letter-policy	5 GiB or 7 days	30 days after rollover

7.5 Certificate lifecycle

- Initial bootstrap creates a CA only when both CA certificate and private key are absent.
- Incomplete CA material causes fail-closed startup; it is never silently replaced.
- Missing service certificates are reissued from the existing CA.
- Trust-root rotation requires `elk/rotate-elastic-ca.sh --confirm-trust-root-rotation`.
- After CA rotation, redistribute the new public CA and verify every client integration.

8. Pre-deployment preparation

8.1 Host prerequisites

- Linux host with Docker Engine and Docker Compose v2.
- OpenSSL for secret generation and certificate operations.
- A trusted HTTPS reverse proxy such as Nginx, Caddy or Traefik.
- DNS control for hirkanet.com and www.hirkanet.com.
- Recommended combined-host baseline: 4 CPU cores, 8 GiB RAM and at least 40 GiB free disk, plus growth capacity.
- Linux `vm.max_map_count=262144` before starting Elasticsearch.

```
docker version
docker compose version
openssl version
sysctl vm.max_map_count
```

8.2 Project location and ownership

```
cd /path/to/hirkanet.com
pwd
ls -la docker-compose.yml docker-compose.elastic.yml .env.example secrets/
```

Run commands from the project root. The deployment account must own the project, secret directory and writable bind-mount directories. Do not use `chmod 777`.

8.3 Create .env

```
cp .env.example .env
chmod 600 .env
nano .env
```

Setting	Example / consideration
POSTGRES_DB	hirkanet
POSTGRES_USER	hirkanet; use a dedicated application database role
BOOTSTRAP_ADMIN_USERNAME	admin or another controlled initial username
APP_ENV	production
LOG_LEVEL	INFO
LOG_FORMAT	json
TRUST_PROXY_HEADERS	true only behind the documented trusted proxy
TRUSTED_HOSTS	hirkanet.com,www.hirkanet.com,localhost,127.0.0.1
STACK_VERSION	Use one verified version across all Elastic images
ES_JAVA_OPTS	Example: <code>-Xms512m -Xmx512m</code>

Setting	Example / consideration
LS_JAVA_OPTS	Example: -Xms256m -Xmx256m

Version check: Confirm the selected Elastic version exists in the official [docker.elastic.co](https://www.docker.elastic.co) registry for Elasticsearch, Logstash, Kibana and Filebeat. Do not mix component versions without explicit compatibility validation.

8.4 Create secret files

The application stack requires the first three files. The Elastic stack requires the remaining six. Example templates are not mounted at runtime.

```
mkdir -p secrets
openssl rand -hex 32 > secrets/postgres_password
openssl rand -hex 32 > secrets/flask_secret_key
openssl rand -hex 32 > secrets/bootstrap_admin_password
openssl rand -hex 32 > secrets/elastic_password
openssl rand -hex 32 > secrets/kibana_password
openssl rand -hex 32 > secrets/logstash_password
openssl rand -hex 32 > secrets/kibana_security_encryption_key
openssl rand -hex 32 > secrets/kibana_saved_objects_encryption_key
openssl rand -hex 32 > secrets/kibana_reporting_encryption_key
chmod 700 secrets
chmod 600 secrets/*
```

Secret handling: Do not commit secret files, paste them into tickets, place them in `.env`, or expose them in command-line arguments. Back them up through an encrypted, access-controlled secret-management process.

8.5 Prepare writable directories

```
mkdir -p data/snapshots uploads static/uploads/blog backups/postgres
# Set ownership to the deployment account and the container-compatible UID/GID.
# Avoid world-writable permissions.
```

8.6 Validate configuration before startup

```
docker compose config
docker compose -f docker-compose.elastic.yml config
find secrets -maxdepth 1 -type f -not -name '*.example' -ls
```

9. Docker deployment

9.1 Application stack startup

```
docker compose pull
docker compose up -d --build
docker compose ps
docker compose logs -f --tail=150 db migrate app
```

16. PostgreSQL starts and passes an authenticated SELECT 1 health check.
17. The migrate service applies all Alembic revisions and seeds defaults.
18. The app starts only after the migration service exits successfully.
19. The application readiness check calls /readyz and validates database access.

9.2 First-start verification

```
curl -fsS http://127.0.0.1:5000/livez
curl -fsS http://127.0.0.1:5000/readyz
docker compose exec db psql -U "$POSTGRES_USER" -d "$POSTGRES_DB" -c 'SELECT 1;'
docker compose run --rm migrate flask --app api.app db current
```

9.3 Elastic stack startup

```
sudo sysctl -w vm.max_map_count=262144
docker compose -f docker-compose.elastic.yml pull
docker compose -f docker-compose.elastic.yml up -d
docker compose -f docker-compose.elastic.yml ps
docker compose -f docker-compose.elastic.yml logs -f --tail=150 elastic-certs-setup elasticsearch
elastic-init logstash kibana filebeat
```

9.4 Resource controls

Service	CPU	Memory limit	PID limit
app	0.75	512 MiB	256
db	1.0	768 MiB	256
elasticsearch	1.5	1536 MiB	512
logstash	1.0	768 MiB	256
kibana	1.0	1 GiB	256
filebeat	0.5	256 MiB	128
setup/init jobs	0.5	512 MiB	128

These are baseline controls, not universal sizing. Monitor actual workload and adjust under change control. Elasticsearch requires memory beyond the JVM heap for native allocations and filesystem cache.

9.5 Independent lifecycle commands

```
# Application only
docker compose stop
docker compose start
docker compose down

# Elastic only
docker compose -f docker-compose.elastic.yml stop
docker compose -f docker-compose.elastic.yml start
docker compose -f docker-compose.elastic.yml down
```

Data deletion: docker compose down does not remove named volumes. Adding -v deletes persistent volumes. Do not use -v on production stacks unless irreversible deletion is explicitly intended and backups are verified.

10. HTTPS and public-domain deployment

10.1 Domain model

URL	Behavior
https://hirkonet.com	Canonical application URL.
https://www.hirkonet.com	Permanent redirect to canonical apex domain.
http://hirkonet.com	Redirect to HTTPS canonical URL.
http://www.hirkonet.com	Redirect to HTTPS canonical URL.

10.2 DNS and firewall

- Create A/AAAA records for hirkonet.com and www.hirkonet.com pointing to the public reverse-proxy address.
- Expose only TCP 80 and 443 publicly for the web service.
- Keep application port 5000 bound to 127.0.0.1.
- Keep PostgreSQL, Elasticsearch and Kibana unexposed or loopback-only as configured.

10.3 Nginx deployment

The supplied deployment/nginx/hirkonet.conf terminates TLS, redirects HTTP and www traffic, forwards requests to 127.0.0.1:5000, sets the expected X-Forwarded headers and enables HSTS after TLS is confirmed.

```
sudo cp deployment/nginx/hirkonet.conf /etc/nginx/sites-available/hirkonet.conf
sudo ln -s /etc/nginx/sites-available/hirkonet.conf /etc/nginx/sites-enabled/hirkonet.conf
sudo nginx -t
sudo systemctl reload nginx
```

10.4 Certificate issuance

```
sudo certbot --nginx -d hirkonet.com -d www.hirkonet.com
sudo certbot renew --dry-run
```

HSTS sequencing: Enable a long HSTS policy only after both domains serve valid HTTPS and renewal has been tested. A premature HSTS deployment can make recovery difficult.

10.5 Proxy trust

```
APP_ENV=production
TRUST_PROXY_HEADERS=true
TRUSTED_HOSTS=hirkonet.com,www.hirkonet.com,localhost,127.0.0.1
```

ProxyFix is configured for exactly one trusted proxy hop. Do not expose Unicorn directly to untrusted clients while accepting forwarded headers, and do not increase the trusted-hop count without understanding the complete proxy chain.

11. User and administrator guide

11.1 Initial administrator

The migration service runs the seed-defaults command after schema upgrade. When no administrator exists, it can create the bootstrap administrator using `BOOTSTRAP_ADMIN_USERNAME` and the mounted `bootstrap_admin_password` secret. Change the bootstrap password immediately after the first secure login.

11.2 Public site

- Home page presents the Hirkanet platform and published services/content.
- Blog listing shows published posts ordered by publication time.
- Blog detail routes resolve published slugs.
- Unpublished content remains in the administrative workflow.

11.3 Client portal

20. Sign in using a PostgreSQL-backed client account.
21. Open the client dashboard and confirm the policy-evaluation subscription is active.
22. Open Policy Evaluation.
23. Enter comma-separated source, destination and service values.
24. Review validation errors before evaluation.
25. Interpret the ranked candidates and scope disclaimer.
26. For bulk work, upload a UTF-8 CSV with source, destination and optional service columns.

```
source,destination,services
192.168.10.25,10.10.10.50,tcp-443
HR-CLIENTS,APP-SERVERS,HTTPS
192.168.20.0/24,10.20.0.0/24,tcp-8000-8080
```

11.4 Administrative portal

Area	Operations
Users	Create, edit and delete users; assign admin or client role.
Subscriptions	Grant or remove access to active services.
Services	Create and activate platform services and service types.
Blog categories	Create, edit and remove categories; posts are retained with category set to NULL.
Blog posts	Draft, publish, unpublish, edit and manage cover images.
Profile	Change password under the shared password policy.

11.5 Safe evaluation workflow

27. Confirm the active snapshot timestamp and target FortiGate/VDOM.
28. Validate the business request, source ownership, destination ownership and required service.
29. Run the evaluation and review all ranked candidates, not only the first row.

30. Open the relevant FortiGate configuration and verify complete policy semantics.
31. Perform risk assessment and peer review.
32. Test through approved methods and maintain a rollback plan.
33. Record the outcome and supporting evidence in change management.

12. Database operations

12.1 Creation and migration

```
# Docker deployment
docker compose up -d db
docker compose run --rm migrate

# Local development
python3 -m flask --app api.app db upgrade
python3 -m flask --app api.app seed-defaults
python3 api/app.py
```

Missing tables: If local execution reports `sqlite3.OperationalError: no such table`, apply flask db upgrade before starting the application. The web process intentionally does not create tables.

12.2 Migration authoring

```
# Local development environment with updated models
flask --app api.app db migrate -m "describe schema change"
# Review the generated revision carefully
flask --app api.app db upgrade
pytest -q
```

Do not generate revisions inside the hardened read-only migration container. Generate locally, review the migration and commit it as source code. The container applies reviewed revisions only.

12.3 Backup

```
./scripts/postgres-backup.sh
RETENTION_DAYS=30 KEEP_MINIMUM=14 ./scripts/postgres-backup.sh
```

The backup script creates a custom-format dump, SHA-256 checksum and metadata file, validates `pg_restore --list`, restores into a disposable test database, verifies expected Hirkanet tables and `alembic_version`, then marks the backup restore_test=passed.

12.4 Test restore and guarded recovery

```
./scripts/postgres-restore-test.sh backups/postgres/<backup>.dump

./scripts/postgres-restore.sh --backup backups/postgres/<backup>.dump --target-db
hirkanet_recovery --confirm-restore
```

Production target: The restore script refuses to overwrite the production database unless `--allow-production-target` is also supplied. Restore into a separate database first whenever practical.

12.5 Database monitoring queries

```
SELECT pg_size_pretty(pg_database_size(current_database()));

SELECT schemaname,
       relname,
       pg_size_pretty(pg_total_relation_size(relid)) AS total_size
FROM pg_catalog.pg_statio_user_tables
ORDER BY pg_total_relation_size(relid) DESC;

SELECT version_num FROM alembic_version;
```

12.6 Operational schedule

Frequency	Actions
Daily	Verified backup, backup-age check, database/volume threshold check, error-log review.
Weekly	Restore test review, table-growth review, failed-login and database-error review.
Monthly	Full recovery drill, retention review, role/subscription audit, capacity forecast.
Before release	Backup, migration review, staging upgrade, restore validation, rollback decision.

13. FortiGate snapshot operations

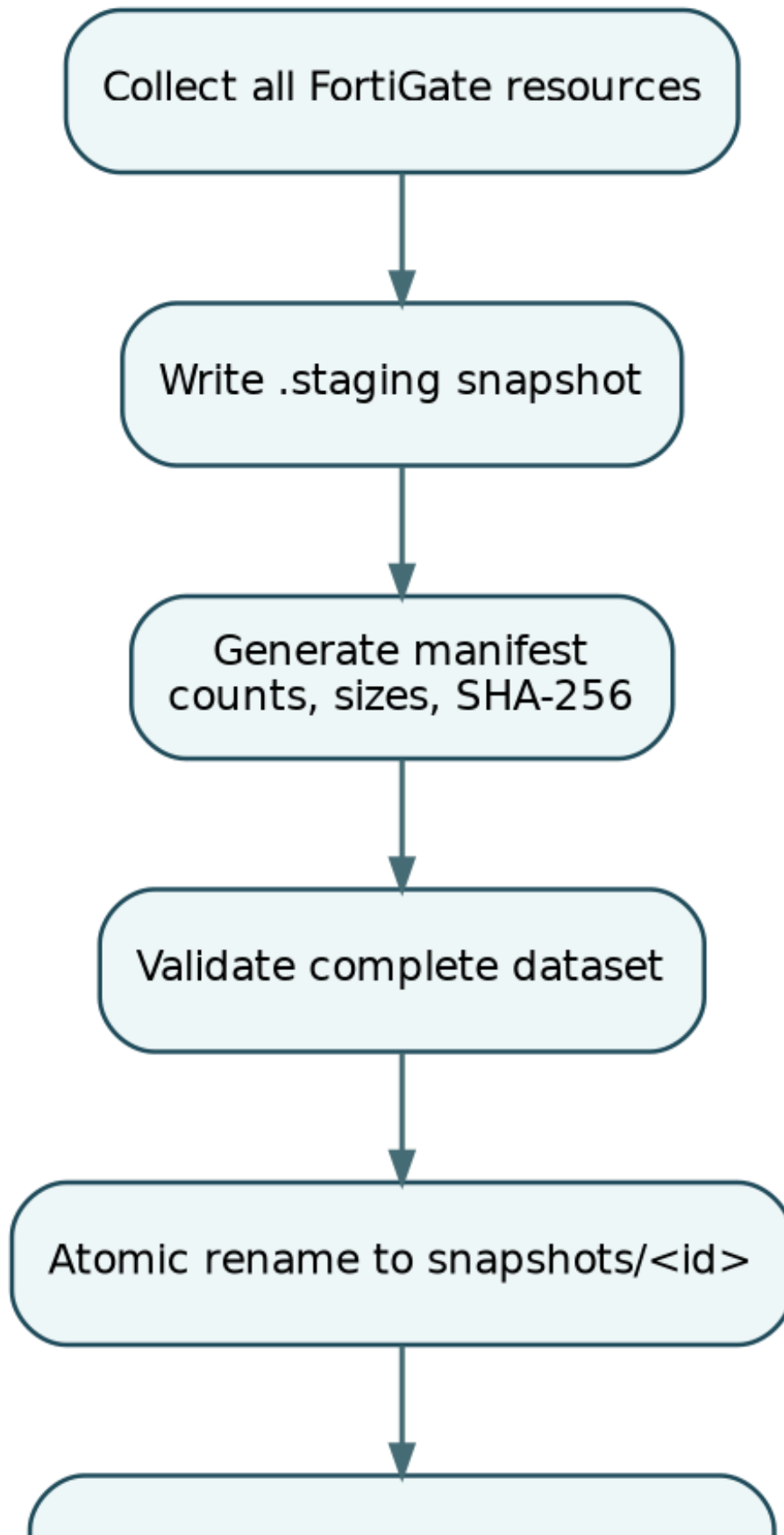


Figure 5 - Atomic FortiGate snapshot publication

13.1 Collector configuration

```
export FORTIGATE_HOST=192.168.1.99
export FORTIGATE_TOKEN='restricted-api-token'
export FORTIGATE_VDOM=root
python collectors/fortigate_collector.py
```

TLS verification: The collector can operate against self-signed FortiGate management certificates, but production should use `--verify` with a trusted certificate whenever possible. Use a restricted REST administrator and protect the token.

13.2 Snapshot contents

```
data/
├── current.json
├── snapshots/
│   └── <snapshot-id>/
│       ├── manifest.json
│       ├── policies.json
│       ├── addresses.json
│       ├── services.json
│       ├── routes.json
│       └── interfaces.json
```

13.3 Atomic publication guarantees

- A lock prevents concurrent collection runs.
- All resources are collected before publication.
- Files are written into a staging directory.
- The manifest records source metadata, counts, sizes and SHA-256 checksums.
- The completed staging directory is atomically renamed.
- The current.json pointer is atomically replaced last.
- A failure before pointer replacement leaves the prior active snapshot intact.

13.4 Retention and rollback

```
python collectors/fortigate_collector.py --keep-snapshots 20
# or
export FORTIGATE_KEEP_SNAPSHOTS=20
```

Rollback is performed by changing current.json to a verified earlier snapshot under controlled procedures. Never edit immutable snapshot files in place. Backup current.json and snapshots together so the pointer and data remain consistent.

14. Monitoring, retention and incident response

14.1 Storage monitor

```
./scripts/storage-monitor.py
./scripts/storage-monitor.py --json
echo $? # 0 healthy, 1 warning/unavailable, 2 critical
```

Metric	Default warning	Default critical
Host filesystem	80%	90%
PostgreSQL volume	80%	90%
PostgreSQL database	5 GiB	10 GiB
Verified backup age	26 hours	48 hours
Backup storage	20 GiB	40 GiB
FortiGate snapshot storage	5 GiB	10 GiB
Upload storage	5 GiB	10 GiB

14.2 Retention

```
# Preview only
./scripts/storage-retention.py

# Apply explicitly
./scripts/storage-retention.py --apply
```

- Expired PostgreSQL backup sets can be removed while preserving the configured minimum count.
- Abandoned partial backup files can be removed.
- Superseded FortiGate snapshots can be removed while protecting the active snapshot.
- Stale staging directories can be removed.
- Blog media is monitored but not automatically deleted.

14.3 Health checks

Service	Readiness signal
PostgreSQL	Authenticated SELECT 1.
Hirkanet	/readyz verifies database; /livez verifies process.
Elasticsearch	Authenticated HTTPS cluster health without timeout and status yellow/green.
Logstash	Node API confirms main pipeline loaded.
Kibana	/api/status reports available.

Service	Readiness signal
Filebeat	Native filebeat test output validates TLS connection to Logstash.

14.4 Incident triage

34. Preserve evidence: timestamps, request IDs, container logs, relevant database state and active snapshot ID.
35. Determine scope: public web, authentication, database, evaluation, collector or Elastic pipeline.
36. Check health endpoints and Compose service status.
37. Review structured logs using `request.id`, `event_type` and `user_id`.
38. Protect data: stop destructive cleanup, confirm backup freshness and snapshot integrity.
39. Apply the smallest reversible remediation.
40. Validate recovery and document root cause, impact and preventive actions.

15. Development, testing and change management

15.1 Local environment

```
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
python3 -m flask --app api.app db upgrade
python3 -m flask --app api.app seed-defaults
python3 api/app.py
```

Development server: `python3 api/app.py` starts Flask debug mode and the Werkzeug development server. Use it only for local development. Production uses Gunicorn behind the HTTPS reverse proxy.

15.2 Tests

```
pytest -q
python -m compileall -q .
docker compose config >/dev/null
docker compose -f docker-compose.elastic.yml config >/dev/null
```

The project includes regression tests for CSRF, validation, IP ranges, TLS, ILM, container hardening, secrets, networks, health checks, migrations, foreign-key actions, constraints, rollback handling, snapshots, backup/restore and storage monitoring.

15.3 Release workflow

41. Review code and configuration changes.
42. Update requirements and migration revisions when dependencies or schema change.
43. Run the full test suite and static configuration validation.
44. Build images and test both Compose stacks in staging.
45. Create and restore-test a PostgreSQL backup.
46. Verify FortiGate snapshot integrity and preserve the active snapshot.
47. Deploy the application stack; confirm migration success before web startup.
48. Deploy the Elastic stack if changed; verify TLS, ILM and health checks.
49. Run smoke tests through the canonical HTTPS domain.
50. Monitor errors, storage and performance, with a documented rollback decision window.

15.4 Dependency management

`requirements.txt` currently contains runtime packages and `pytest`. A future cleanup should split production and development dependencies into `requirements.txt` and `requirements-dev.txt`. Pin and review direct and transitive dependencies, and rebuild images regularly for base-image and OS security updates.

16. Troubleshooting

16.1 Missing .env or blank Compose variables

```
cp .env.example .env
chmod 600 .env
nano .env
docker compose config
```

16.2 Missing secret bind mount

Error: invalid mount config ... secrets/postgres_password does not exist.

```
openssl rand -hex 32 > secrets/postgres_password
openssl rand -hex 32 > secrets/flask_secret_key
openssl rand -hex 32 > secrets/bootstrap_admin_password
chmod 600 secrets/postgres_password secrets/flask_secret_key secrets/bootstrap_admin_password
```

16.3 Missing database table

Error: sqlite3.OperationalError: no such table: blog_posts.

```
python3 -m flask --app api.app db upgrade
python3 -m flask --app api.app seed-defaults
python3 api/app.py
```

16.4 Filebeat image pull failure

Use the official Elastic registry and a version that actually exists for every component.

```
docker.elastic.co/elasticsearch/elasticsearch:${STACK_VERSION}
docker.elastic.co/logstash/logstash:${STACK_VERSION}
docker.elastic.co/kibana/kibana:${STACK_VERSION}
docker.elastic.co/beats/filebeat:${STACK_VERSION}
```

16.5 Migration service fails

```
docker compose logs --tail=200 db migrate
docker compose run --rm migrate flask --app api.app db current
docker compose run --rm migrate flask --app api.app db history
```

- Confirm PostgreSQL is healthy and credentials match the secret file.
- Read the Alembic error; the constraint migration intentionally stops on invalid or duplicate production data.
- Do not bypass a failed migration by starting Gunicorn against an incompatible schema.

16.6 Evaluator reports no data

```
cat data/current.json
find data/snapshots -maxdepth 2 -type f -name manifest.json -print
```

```
python collectors/fortigate_collector.py --host <host> --token <token> --vdom root
```

16.7 Elastic certificate failure

```
docker compose -f docker-compose.elastic.yml logs elastic-certs-setup
# If CA material is incomplete, restore it from backup or use the explicit rotation procedure.
./elk/rotate-elastic-ca.sh --confirm-trust-root-rotation
```

Do not weaken TLS: A hostname mismatch must be fixed by correcting the endpoint or reissuing the affected service certificate with the required SAN. Do not switch verification from full to certificate or none.

Appendix A. Command reference

A.1 Application stack

```
docker compose config
docker compose up -d --build
docker compose ps
docker compose logs -f --tail=150 db migrate app
docker compose down
```

A.2 Elastic stack

```
docker compose -f docker-compose.elastic.yml config
docker compose -f docker-compose.elastic.yml up -d
docker compose -f docker-compose.elastic.yml ps
docker compose -f docker-compose.elastic.yml logs -f --tail=150
docker compose -f docker-compose.elastic.yml down
```

A.3 Database

```
docker compose run --rm migrate
./scripts/postgres-backup.sh
./scripts/postgres-restore-test.sh backups/postgres/<backup>.dump
./scripts/postgres-restore.sh --backup <backup>.dump --target-db hirkanet_recovery --confirm-restore
```

A.4 FortiGate

```
python collectors/fortigate_collector.py --host 192.168.1.99 --token 'token' --vdom root --
verify --keep-snapshots 20
```

A.5 Monitoring

```
./scripts/storage-monitor.py
./scripts/storage-monitor.py --json
./scripts/storage-retention.py
./scripts/storage-retention.py --apply
```

Appendix B. Configuration reference

B.1 Non-secret environment settings

Variable	Meaning
POSTGRES_DB	Application database name.
POSTGRES_USER	Application database role.
POSTGRES_HOST	Database service DNS name; normally db in Compose.
POSTGRES_PORT	Database port; normally 5432.
BOOTSTRAP_ADMIN_USERNAME	Initial administrator username.
APP_ENV	development or production behavior.
SERVICE_NAME	Structured log service.name.
LOG_LEVEL	Application logging threshold.
LOG_FORMAT	json for production observability.
TRUST_PROXY_HEADERS	Enables exactly one trusted proxy hop.
TRUSTED_HOSTS	Accepted Host headers.
STACK_VERSION	Single Elastic component image version.
ES_JAVA_OPTS	Elasticsearch JVM heap.
LS_JAVA_OPTS	Logstash JVM heap.
FORTIGATE_KEEP_SNAPSHOTS	Snapshot retention count.

B.2 Secret files

File	Consumer
secrets/postgres_password	PostgreSQL, migration job and application.
secrets/flask_secret_key	Flask session and CSRF signing.
secrets/bootstrap_admin_password	Initial administrator creation.
secrets/elastic_password	Elasticsearch superuser/bootstrap operations.
secrets/kibana_password	kibana_system account.
secrets/logstash_password	logstash_internal account.
secrets/kibana_security_encryption_key	Kibana security encryption.
secrets/kibana_saved_objects_encryption_key	Kibana saved objects encryption.
secrets/kibana_reporting_encryption_key	Kibana reporting encryption.

Appendix C. Security and production checklist

- ☐ Real .env exists, is mode 0600 and contains no passwords or encryption keys.
- ☐ All required secret files exist, are strong, are mode 0600 and are backed up securely.
- ☐ POSTGRES_USER is a dedicated application role and database backups are restore-tested.
- ☐ All Alembic revisions are applied and the migration job exits successfully.
- ☐ Application source is read-only inside the container; capabilities are dropped.
- ☐ Only ports 80/443 are publicly exposed; port 5000 remains loopback-only.
- ☐ hirkanet.com has a valid certificate; www and HTTP redirect to the canonical HTTPS URL.
- ☐ Proxy headers are trusted only from the intended one-hop reverse proxy.
- ☐ CSRF protection, secure cookies and trusted-host checks are enabled in production.
- ☐ The active FortiGate snapshot is current, verified and backed up.
- ☐ The collector uses a restricted token and verified TLS where possible.
- ☐ Elastic uses official images, one verified version, full hostname verification and bounded ILM retention.
- ☐ Elastic CA rotation is explicit and client trust redistribution is planned.
- ☐ Storage monitoring is scheduled and alert exit codes are integrated with operations.
- ☐ Retention is dry-run reviewed before --apply.
- ☐ Application and database logs are monitored without exposing secrets.
- ☐ A rollback plan exists for application, migration, database, snapshot and certificate changes.

Final operational principle

Control before convenience: Hirkanet is a security platform. Prefer explicit, auditable and reversible operations: validate inputs, keep one source of truth, migrate schemas deliberately, back up and restore-test data, publish snapshots atomically, use full TLS verification, and never treat heuristic policy recommendations as runtime proof.